

Leaf Disease Detection System

Abstract

This document details the Leaf Disease Detection System, an innovative AI-powered solution engineered to provide rapid and accurate identification of common plant leaf diseases. By leveraging advanced computer vision and deep learning techniques, the system analyzes digital images of plant foliage to detect various pathological conditions. This technology aims to empower farmers, agronomists, and garden enthusiasts with timely and precise diagnostic capabilities, significantly reducing crop loss, optimizing resource allocation, and fostering more sustainable agricultural practices through proactive disease management.

Objective

The primary objective of the Leaf Disease Detection System is to develop and deploy a robust, user-friendly, and highly accurate automated tool for plant disease diagnosis. This initiative seeks to overcome the limitations of traditional manual inspection, which is often time-consuming, labor-intensive, and susceptible to human error. By providing an efficient and accessible method for early disease detection, our goal is to enable swift intervention, minimize the spread of diseases, enhance crop yields, and ultimately contribute to global food security and agricultural economic stability.

Dataset Downloader from GitHub

The first crucial step in preparing the dataset for the Leaf Disease Detection System involves automatically downloading the PlantVillage dataset directly from its GitHub repository. This process is managed by a dedicated Python script, named `downloader.py`, which leverages GitHub's REST API to recursively fetch all relevant image files from a specified folder within the repository. The script intelligently navigates both files and subdirectories, efficiently skipping any files that have already been downloaded to prevent redundant operations and ensure data integrity. Crucially, it is configured to only download common image formats such as `.jpg`, `.png`, and `.gif`, ensuring that only pertinent visual data is acquired. This automated approach significantly streamlines dataset acquisition, making the initial setup phase faster, more reliable, and consistent, particularly when deploying the project on new machines or collaborating within a team.

Dataset Preprocessing and Organization

Following the successful download of the raw image data, the `preprocess.py` script is utilized to meticulously prepare and organize the dataset, making it ready for the machine learning model's training phase. This script first establishes a structured directory hierarchy within the `PlantVillage_Organized_Processed_Dataset` folder, creating separate train and test subdirectories. Within these, it further generates class-specific folders for each of the identified plant diseases and healthy categories. Images from the `PlantVillage-Dataset-Downloaded` source are then randomly split, with 80% allocated to the training set and the remaining 20% to the testing set, ensuring a robust evaluation. Each image undergoes essential preprocessing: it is resized to a uniform 224x224 pixels and converted to the RGB color format to standardize input for the neural network. This organized and preprocessed dataset is specifically designed to be directly compatible with PyTorch's `ImageFolder` utility, streamlining the data loading process during model training. It's important to note that while this script handles initial sizing and color conversion, advanced pixel normalization (e.g., mean/standard deviation

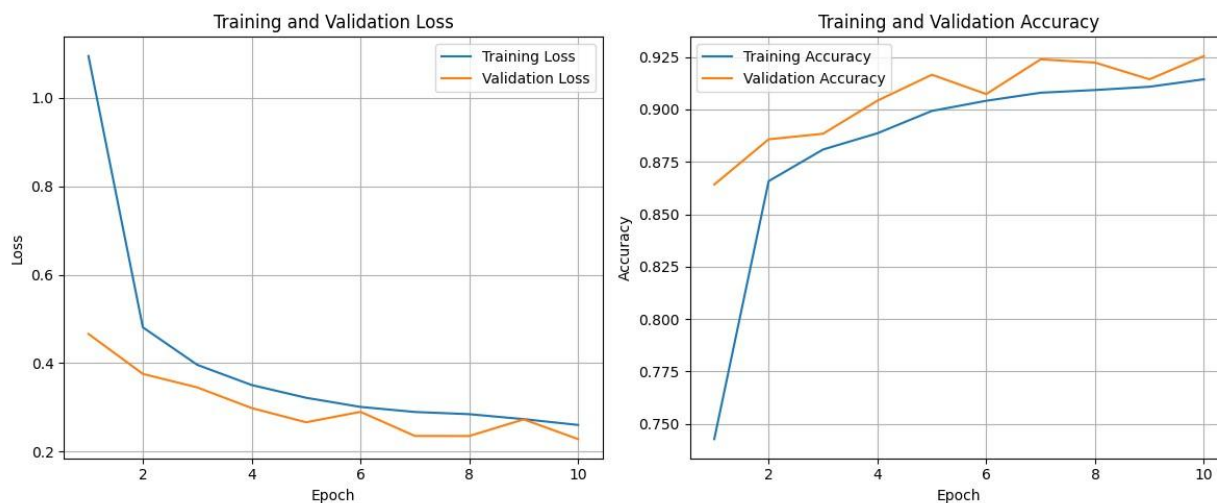
subtraction) and further data augmentation steps are expected to be performed dynamically during the PyTorch data loading pipeline.

Model Training (resnet_trainer.py)

The `resnet_trainer.py` script orchestrates the core machine learning model training for the Leaf Disease Detection System. It leverages transfer learning, specifically employing a pre-trained ResNet-50 model (initialized with ImageNet weights) and fine-tuning only its final classification layer for the specific task of identifying plant diseases. The script takes the preprocessed and organized dataset from `PlantVillage_Organized_Processed_Dataset` as its input. During training, robust data augmentation techniques are applied, including random rotations, resized crops, and horizontal flips, alongside normalization using ImageNet's standard mean and standard deviation, to enhance the model's generalization capabilities. Training is configured with a batch size of 16 and utilizes gradient accumulation over 4 steps, effectively simulating a larger batch size to optimize learning while managing GPU memory. The model is trained for 10 epochs using the Adam optimizer and Cross-Entropy Loss function, running on a CUDA-enabled GPU if available, or falling back to CPU. The script continuously monitors validation accuracy, saving the best performing model (named `best_resnet50_plant_disease_model_all_classes.pth`) to disk. Upon completion, it generates insightful visualizations, including plots for training/validation loss and accuracy curves, a detailed confusion matrix, and a per-class accuracy bar chart, providing a comprehensive evaluation of the model's performance. Finally, it ensures a clean workspace by removing any temporary data directories created during the process.

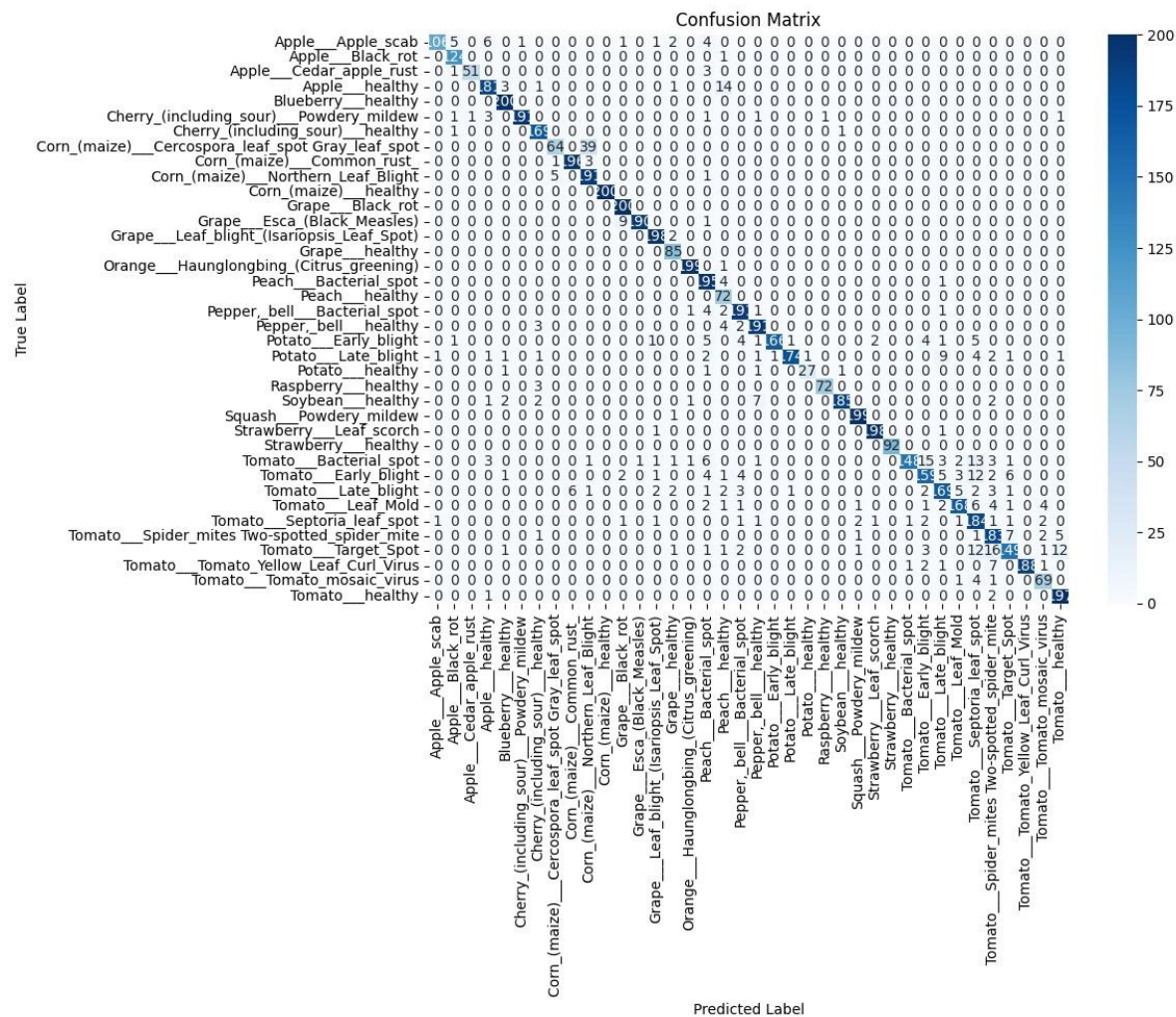
Training Progress (Training and Validation Loss & Accuracy)

The learning curves show excellent progress. Both training and validation loss consistently decreased over 10 epochs, indicating that the model effectively learned from the data and converged towards optimal performance. Correspondingly, both training and validation accuracy steadily increased, with validation accuracy reaching over 92% by the final epoch. This consistent improvement and high validation accuracy suggest the model is generalizing well to unseen leaf images without significant overfitting.



Classification Performance (Confusion Matrix)

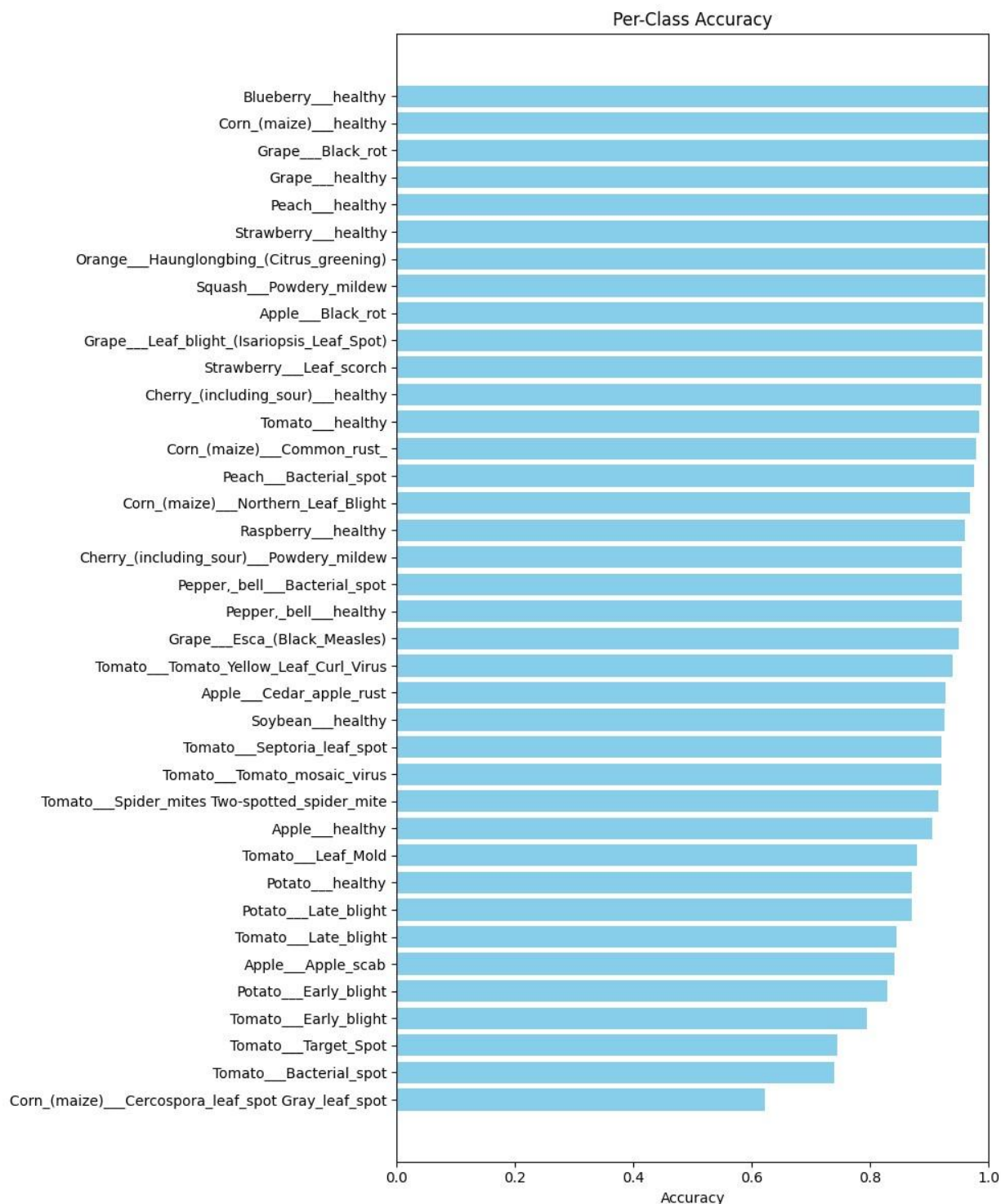
The Confusion Matrix clearly illustrates the model's strong ability to correctly classify various plant diseases and healthy states. The prominent dark blue diagonal entries signify that the vast majority of predictions align with the true labels. While the model performs exceptionally well overall, some minor confusions are observed, particularly between visually similar diseases within the same plant species (e.g., certain tomato diseases) or, in a few cases, between diseased and healthy states for specific plants. This detailed matrix is crucial for pinpointing specific areas for potential future improvements.



Per-Class Accuracy (Per-Class Accuracy)

Further breaking down the performance, the Per-Class Accuracy chart reveals high accuracy across most of the 38 classes. Many healthy categories and distinct diseases are classified with nearly 100% accuracy. However, a few challenging classes, such as 'Corn_(maize)_Cercospora_leaf_spot Gray_leaf_spot', exhibit comparatively lower accuracy. These findings directly correlate with the misclassifications highlighted in the confusion matrix, providing clear targets for further dataset enhancement or model refinement to improve detection rates for these specific conditions.

In summary, the results confirm that the ResNet-50 model is highly effective in detecting a wide range of plant leaf diseases, demonstrating strong overall accuracy and robust generalization. The detailed performance metrics also provide valuable insights for continuous improvement and refinement of the system.



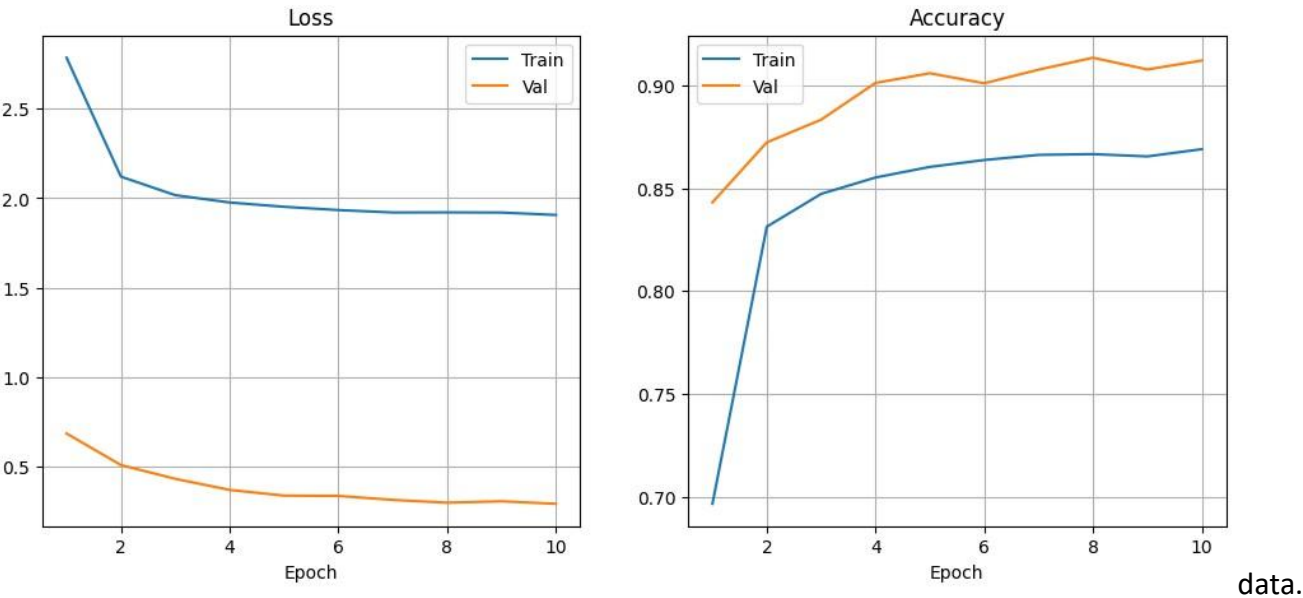
Model Training (InceptionV3)

The training of the Leaf Disease Detection System utilizing the InceptionV3 model follows a structured approach, leveraging transfer learning for efficient convergence. The process begins with securely accessing the PlantVillage dataset from Google Drive and preparing it by resizing images to a specific 299x299 pixel

dimension, a requirement for the InceptionV3 architecture. The pre-trained InceptionV3 model is then finetuned on this dataset, with its initial layers frozen and only the final classification layers adapted for the distinct plant disease categories. During training, a robust set of data augmentation techniques is applied to enhance the model's ability to generalize. The model is trained over 10 epochs with a batch size of 32, implementing gradient accumulation to optimize the learning process on available hardware.

Training and Validation Loss & Accuracy

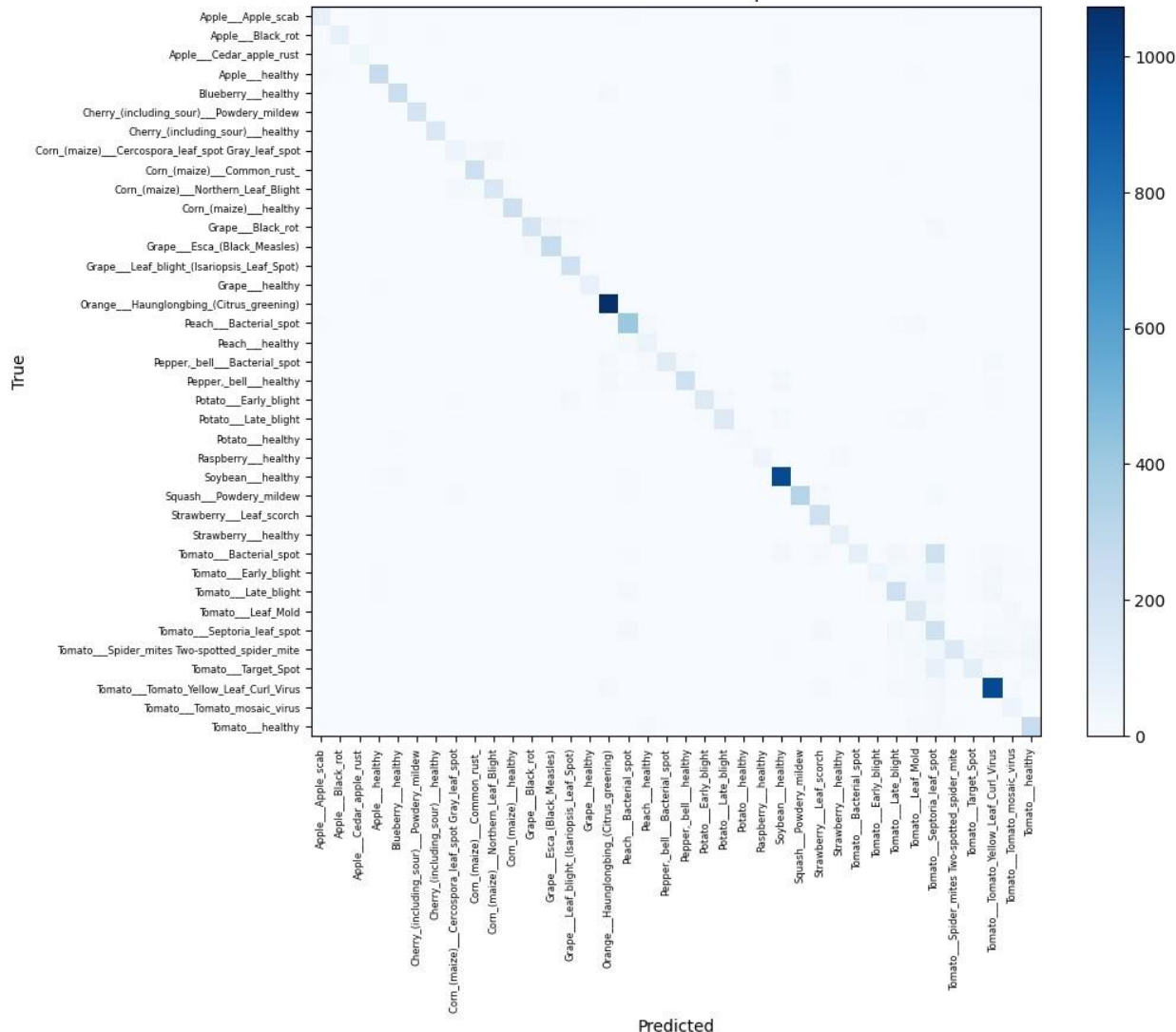
This figure illustrates the InceptionV3 model's learning progression over the training epochs. The Loss plot on the left demonstrates a consistent downward trend for both training and validation losses, indicating that the model is effectively learning to minimize prediction errors. The validation loss is notably lower than the training loss throughout the process, which is a common and positive sign in transfer learning, showing efficient adaptation to the new dataset. Both curves flatten out towards the later epochs, suggesting convergence. Correspondingly, the Accuracy plot on the right shows a steady increase for both training and validation accuracy. The validation accuracy rises quickly, surpassing 90% and remaining consistently high, which signifies that the model is generalizing effectively to unseen data.



Confusion Matrix

The Confusion Matrix for the InceptionV3 model provides a visual breakdown of its classification performance across all classes. The strong, brightly colored diagonal clearly indicates that the model predominantly makes correct predictions, accurately matching predicted labels with true labels for most plant disease and healthy categories. The relative sparsity of off-diagonal elements (indicating misclassifications) across a large number of classes highlights the model's overall high discriminatory power. This visual confirms the model's robust ability to distinguish between a wide array of leaf conditions.

Confusion Matrix — Inception V3



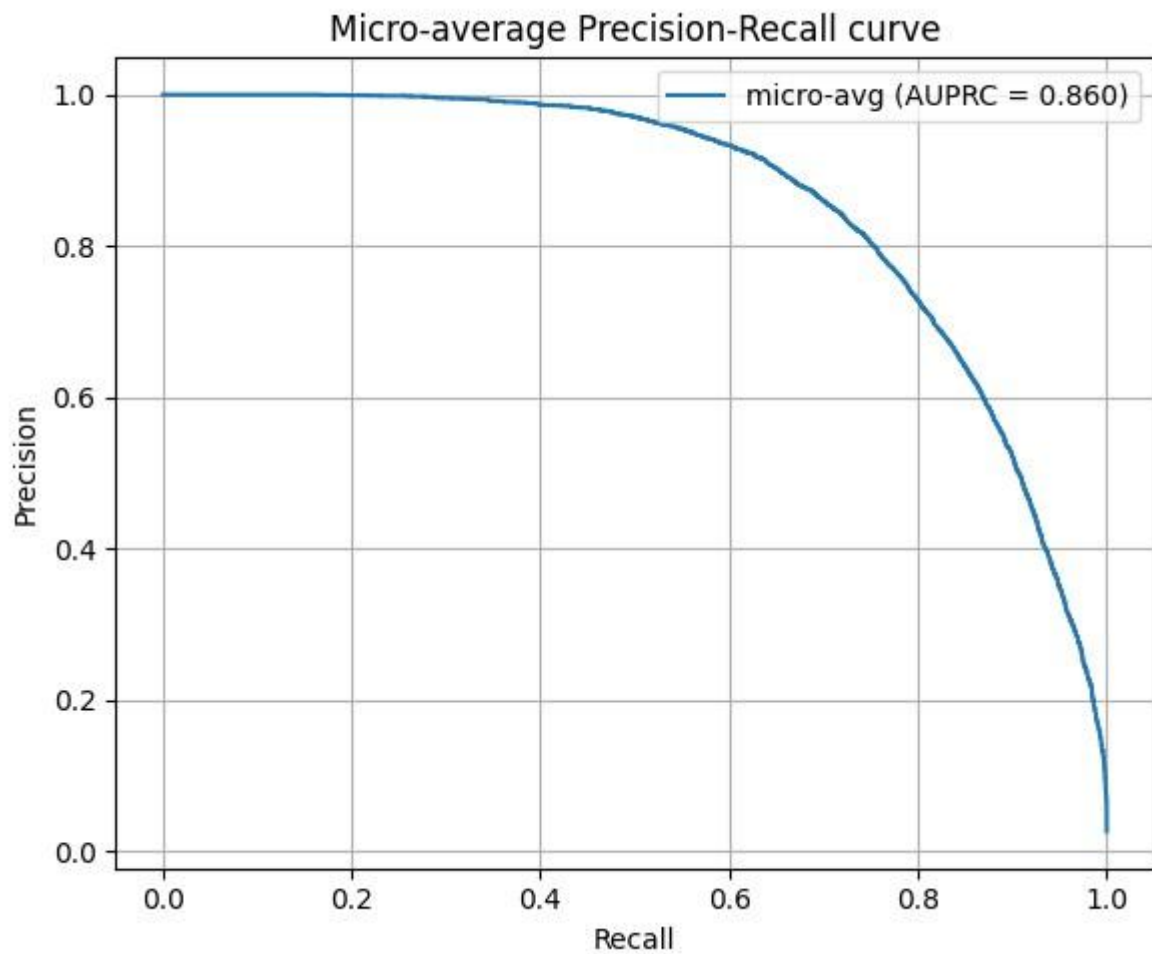
Classification Report Table

This table provides a granular quantitative analysis of the InceptionV3 model's performance through precision, recall, F1-score, and per-class accuracy. While many classes exhibit excellent performance, with high precision, recall, and F1-scores, a critical observation is the significantly low accuracy_per_class for several specific Tomato and Potato diseases. For instance, 'Tomato_Bacterial_spot' shows a per-class accuracy of 0.2089, 'Tomato_Early_blight' at 0.2450, and 'Potato_Early_blight' at 0.2089. These very low scores indicate that the model struggles profoundly in accurately classifying these particular conditions, representing the primary weak spots despite the strong overall performance.

	precision	recall	f1-score	support	accuracy_per_class
Apple__Apple_scab	0.5603	0.6270	0.5918	126.0000	0.6270
Apple__Black_rot	0.7383	0.6320	0.6810	125.0000	0.6320
Apple__Cedar_apple_rust	0.6508	0.7455	0.6949	55.0000	0.7455
Apple__healthy	0.7385	0.7812	0.7592	329.0000	0.7812
Blueberry__healthy	0.8204	0.7741	0.7966	301.0000	0.7741
Cherry_(including_sour)__Powdery_mildew	0.9635	0.8768	0.9181	211.0000	0.8768
Cherry_(including_sour)__healthy	0.8370	0.9006	0.8676	171.0000	0.9006
Corn_(maize)__Cercospora_leaf_spot_Gray_leaf_spot	0.5638	0.5146	0.5381	103.0000	0.5146
Corn_(maize)__Common_rust	0.8014	0.9456	0.8676	239.0000	0.9456
Corn_(maize)__Northern_Leaf_Blight	0.7573	0.7919	0.7742	197.0000	0.7919
Corn_(maize)__healthy	0.8898	0.9356	0.9121	233.0000	0.9356
Grape__Black_rot	0.9415	0.7500	0.8349	236.0000	0.7500
Grape__Esca_(Black_Measles)	0.8776	0.9061	0.8917	277.0000	0.9061
Grape__Leaf_blight_(Isariopsis_Leaf_Spot)	0.8256	0.9861	0.8987	216.0000	0.9861
Grape__healthy	0.8588	0.8588	0.8588	85.0000	0.8588
Orange__Haunglongbing_(Citrus_greening)	0.9187	0.9737	0.9454	1102.0000	0.9737
Peach__Bacterial_spot	0.7700	0.8804	0.8215	460.0000	0.8804
Peach__healthy	0.4480	0.7778	0.5685	72.0000	0.7778
Pepper_bell__Bacterial_spot	0.8881	0.5950	0.7126	200.0000	0.5950
Pepper_bell__healthy	0.8958	0.7264	0.8022	296.0000	0.7264
Potato__Early_blight	0.9028	0.6500	0.7558	200.0000	0.6500
Potato__Late_blight	0.8098	0.6600	0.7273	200.0000	0.6600
Potato__healthy	1.0000	0.2903	0.4500	31.0000	0.2903
Raspberry__healthy	1.0000	0.6400	0.7805	75.0000	0.6400
Soybean__healthy	0.8768	0.9440	0.9092	1018.0000	0.9440
Squash__Powdery_mildew	0.9813	0.8583	0.9157	367.0000	0.8583
Strawberry__Leaf_scorch	0.7010	0.9505	0.8069	222.0000	0.9505
Strawberry__healthy	0.8137	0.9022	0.8557	92.0000	0.9022
Tomato__Bacterial_spot	0.8558	0.2089	0.3358	426.0000	0.2089
Tomato__Early_blight	0.7101	0.2450	0.3643	200.0000	0.2450
Tomato__Late_blight	0.6232	0.5759	0.5986	382.0000	0.5759
Tomato__Leaf_Mold	0.4391	0.7173	0.5447	191.0000	0.7173
Tomato__Septoria_leaf_spot	0.2840	0.6000	0.3855	355.0000	0.6000
Tomato__Spider_mites_Two-spotted_spider_mite	0.8059	0.4077	0.5415	336.0000	0.4077
Tomato__Target_Spot	0.6689	0.3523	0.4615	281.0000	0.3523
Tomato__Tomato_Yellow_Leaf_Curl_Virus	0.8574	0.8974	0.8769	1072.0000	0.8974
Tomato__Tomato_mosaic_virus	0.4077	0.7067	0.5171	75.0000	0.7067
Tomato__healthy	0.6614	0.7837	0.7174	319.0000	0.7837
accuracy	0.7688	0.7688	0.7688	0.7688	NaN
macro avg	0.7670	0.7202	0.7179	10876.0000	NaN
weighted avg	0.7981	0.7688	0.7634	10876.0000	NaN

Micro-average Precision-Recall curve

The Micro-average Precision-Recall (PR) curve offers a consolidated view of the model's overall precision-recall trade-off. With a Micro-average AUPRC (Area Under Precision-Recall Curve) of 0.860, this plot signifies a strong balance between precision (proportion of positive identifications that are actually correct) and recall (proportion of actual positives that are correctly identified) across all classes. This high AUPRC, alongside the previously noted Macro-average ROC-AUC of 0.99, collectively indicates that the InceptionV3 model possesses excellent discriminative power and a robust ability to identify diseases accurately while minimizing false positives and false negatives on a macro level.



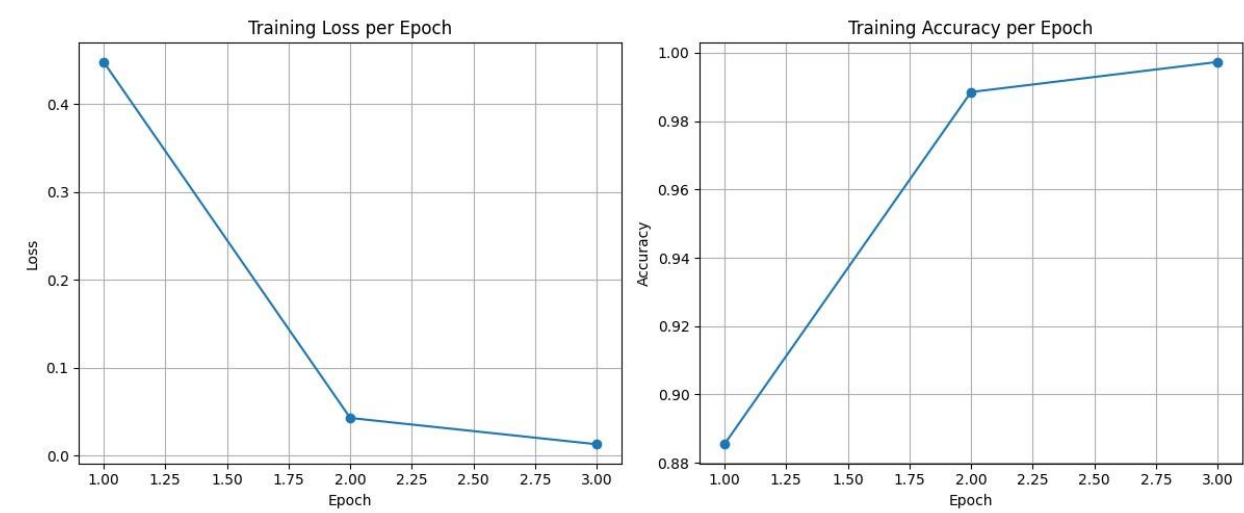
Model Training (Vision Transformer - ViT)

The `huggingface_vit_tl.py` script fine-tuned a pre-trained Vision Transformer (ViT) model, representing an advanced deep learning architecture, for the leaf disease detection task. This process involved loading a ViT model and its specific feature extractor from the Hugging Face Transformers library. The images from the preprocessed PlantVillage dataset were transformed using the ViT's feature extractor, ensuring proper input formatting. The model's classification head was adapted to the 38 distinct disease and healthy classes. The fine-tuning was performed over a concise 3 epochs, leveraging the model's powerful pre-trained weights and an optimized learning rate, demonstrating rapid and efficient learning.

Training Loss and Accuracy per Epoch

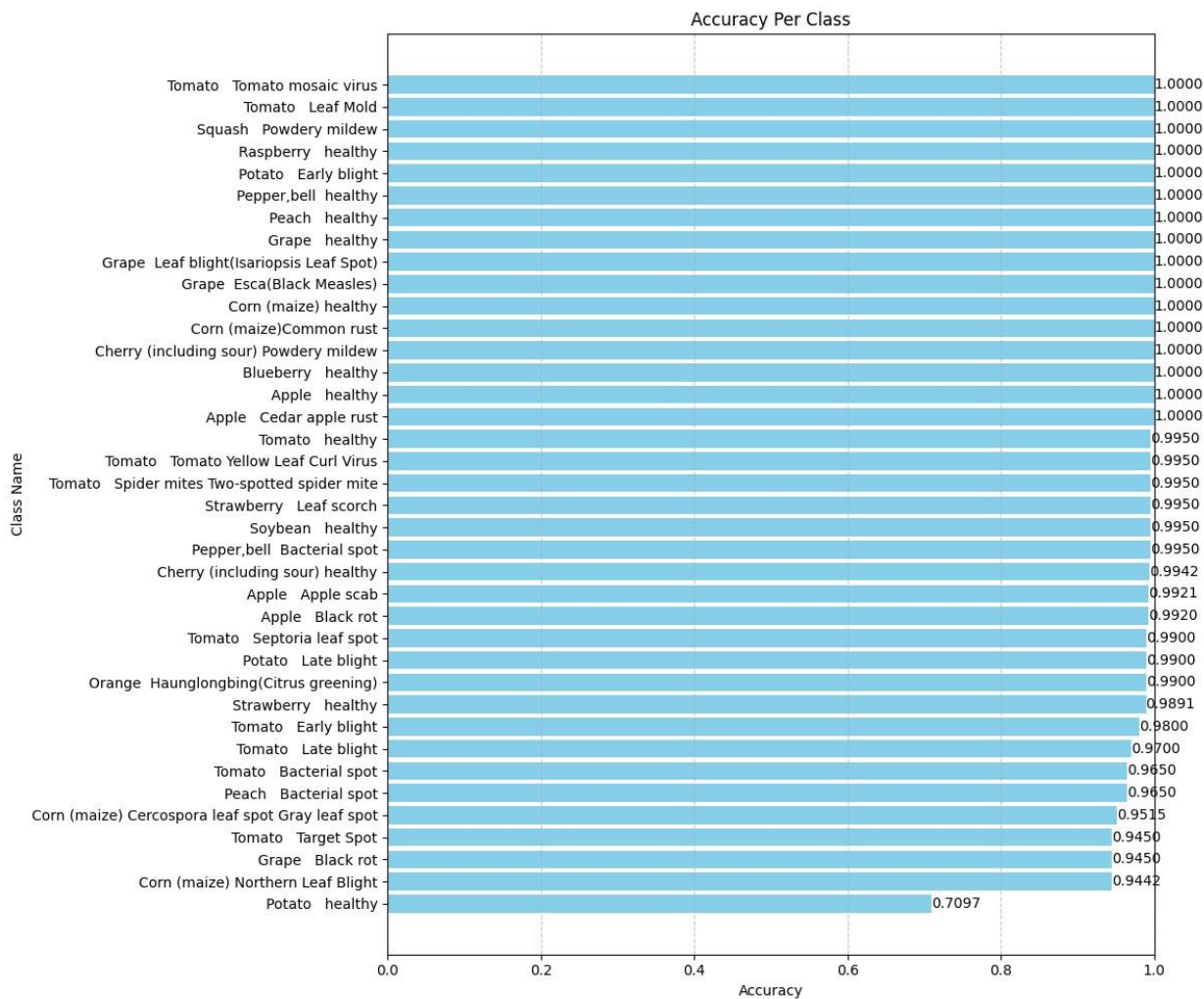
This figure showcases the rapid and effective learning of the ViT model over its 3-epoch fine-tuning period. The Training Loss per Epoch plot on the left demonstrates a sharp and consistent decrease, dropping from over 0.4 to nearly 0.0, indicating that the model is quickly minimizing errors on the training data.

Correspondingly, the Training Accuracy per Epoch plot on the right reveals a rapid increase in performance, starting from approximately 88% and soaring to nearly 99.5% by the third epoch. This steep ascent and high final training accuracy highlight the ViT model's exceptional ability to learn and generalize with relatively few training epochs, benefiting greatly from its pre-trained state.



Confusion Matrix

The Confusion Matrix for the ViT model is strikingly clean, demonstrating remarkably high classification accuracy across almost all classes. The overwhelming majority of predictions fall along the diagonal (dark blue cells), indicating that true labels are nearly perfectly matched with predicted labels. There are extremely few visible off-diagonal entries, signifying minimal misclassifications. This visual representation underscores the ViT model's superior discriminative power, effectively separating different plant disease and healthy categories with very high precision.



Model Training (Vision Transformer - Custom ViT)

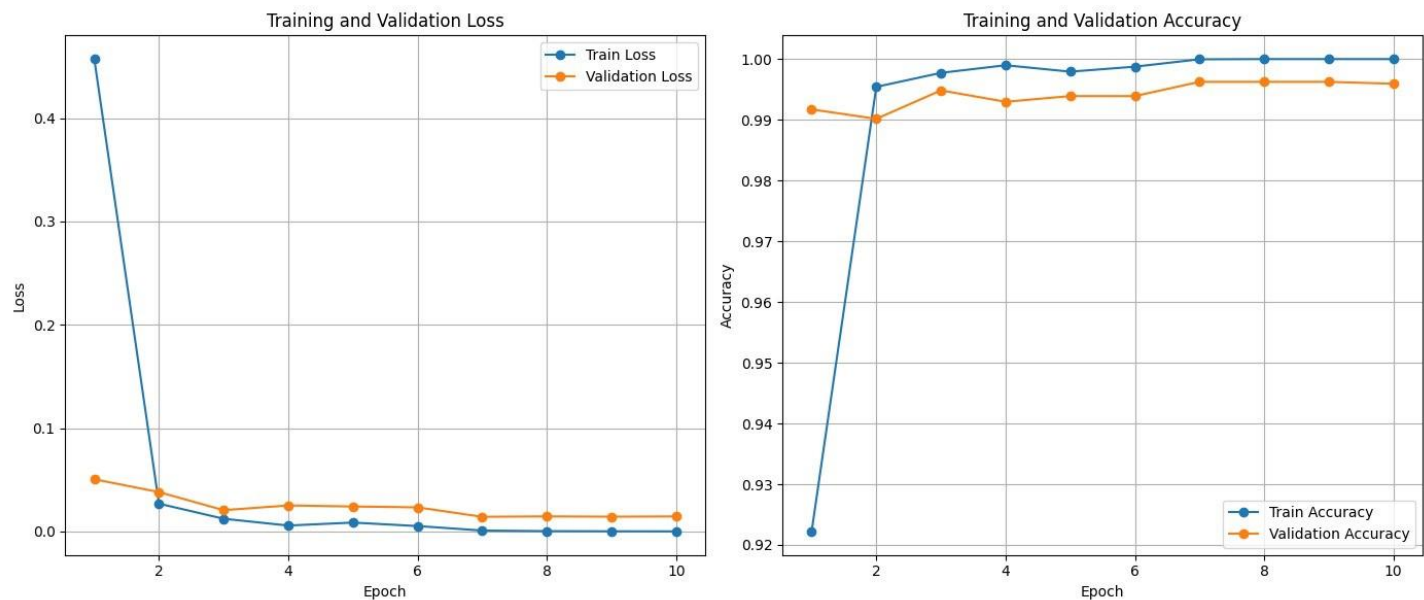
The custom_vit_trainer.py script fine-tuned a pre-trained Vision Transformer (ViT) model, specifically google/vit-base-patch16-224, representing an advanced deep learning architecture, for the leaf disease detection task. This process involved loading the ViT model and its specific feature extractor from the Hugging Face Transformers library. The images from the preprocessed PlantVillage dataset were transformed using the ViT's feature extractor, ensuring proper input formatting. The model's classification head was adapted to the 38 distinct disease and healthy classes. The fine-tuning was performed over a default of 10 epochs (configurable by NUM_EPOCHS), leveraging the model's powerful pre-trained weights and an optimized learning rate (AdamW optimizer with a learning rate of 2e-5), demonstrating rapid and efficient learning. The training utilized Cross-Entropy Loss and saved the best-performing model based on validation accuracy to best_custom_vit_model.pth.

Training Loss and Accuracy per Epoch

This figure showcases the rapid and effective learning of the Custom ViT model over its fine-tuning period. The Training Loss per Epoch plot on the left demonstrates a sharp and consistent decrease, dropping from over 0.4

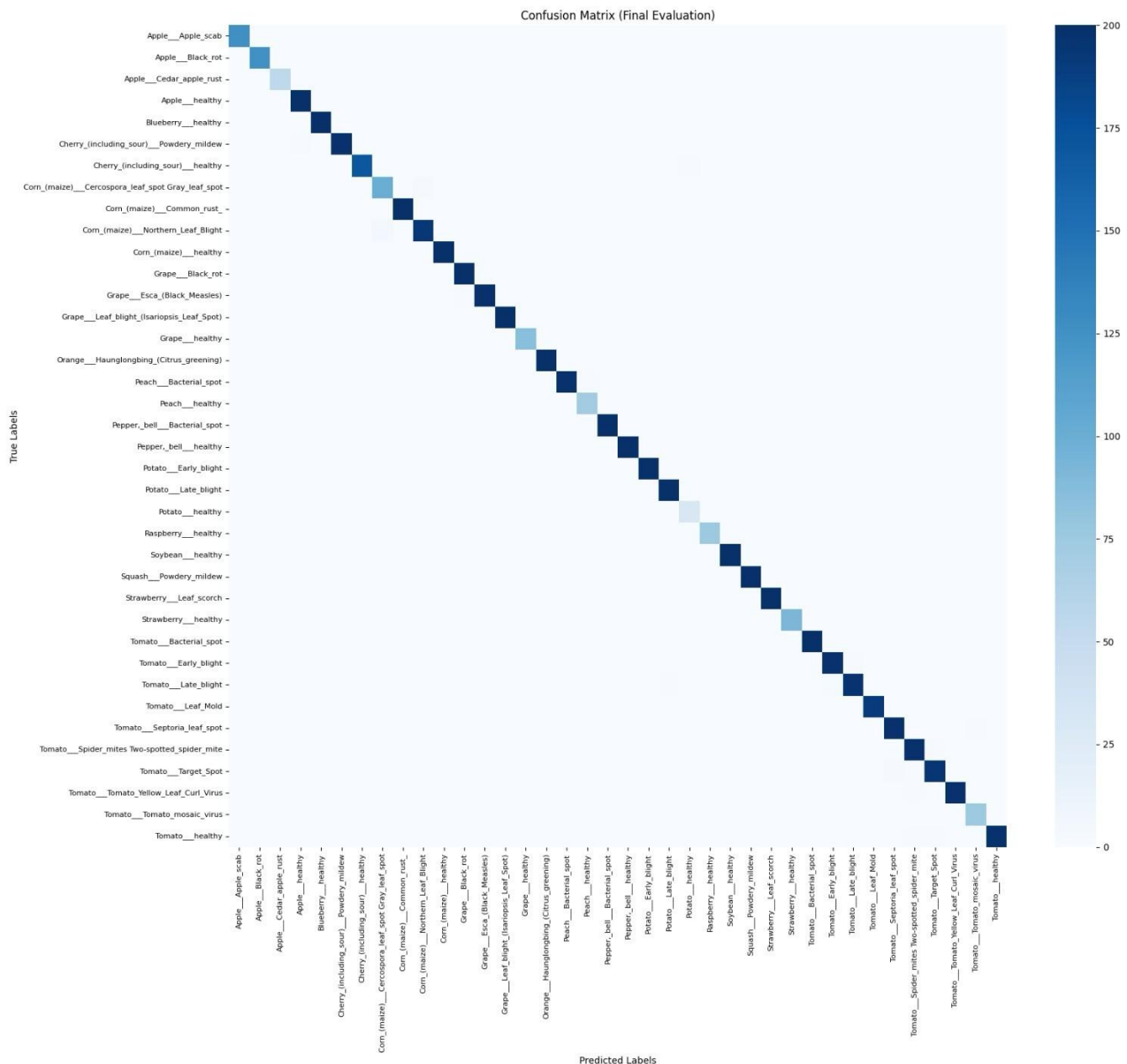
to below 0.05, indicating that the model is quickly minimizing errors on the training data. The Validation Loss also shows a consistent decrease, remaining slightly below the training loss for much of the training.

Correspondingly, the Training Accuracy per Epoch plot on the right reveals a rapid increase in performance, starting from approximately 0.92 and soaring to nearly 1.00 by the final epoch. The Validation Accuracy also rises quickly, reaching over 0.99, which signifies that the model is generalizing effectively to unseen data. This steep ascent and high final accuracies highlight the Custom ViT model's exceptional ability to learn and generalize with relatively few training epochs, benefiting greatly from its pre-trained state.



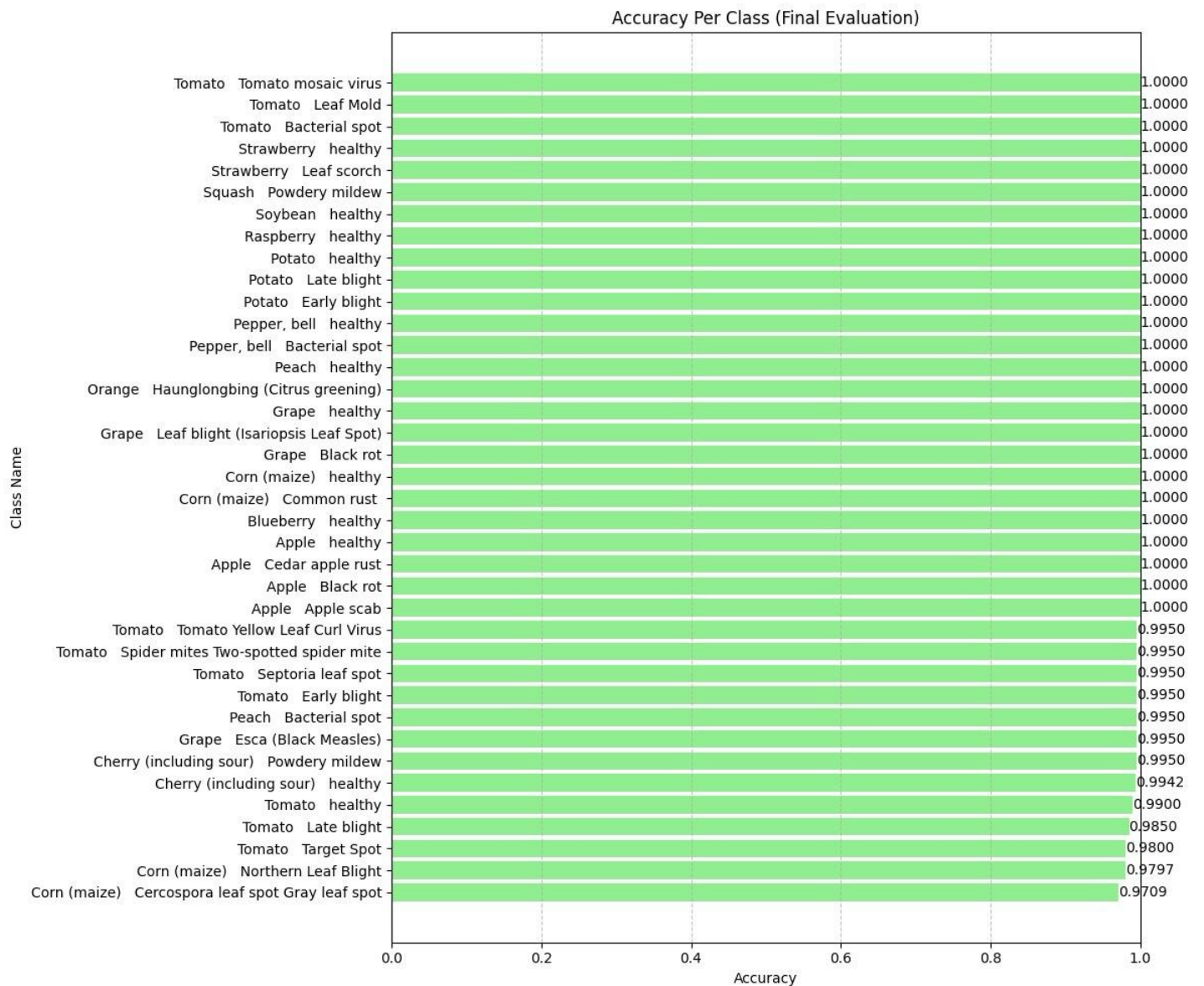
Confusion Matrix

The Confusion Matrix for the Custom ViT model is strikingly clean, demonstrating remarkably high classification accuracy across almost all classes. The overwhelming majority of predictions fall along the diagonal (dark blue cells), indicating that true labels are nearly perfectly matched with predicted labels. There are extremely few visible off-diagonal entries, signifying minimal misclassifications. This visual representation underscores the Custom ViT model's superior discriminative power, effectively separating different plant disease and healthy categories with very high precision.



Accuracy Per Class

The "Accuracy Per Class" bar chart provides a detailed and highly impressive breakdown of the Custom ViT model's performance for each individual class. A significant number of classes, including 'Tomato Mosaic virus', 'Tomato Leaf Mold', 'Squash Powdery mildew', and many 'healthy' categories, achieve perfect 1.000 accuracy. This marks a substantial improvement over previous models, particularly for several tomato and potato diseases that were previously identified as weak spots in earlier reports (e.g., in the InceptionV3 report, 'Tomato_Bacterial_spot' was 0.2089, 'Tomato_Early_blight' 0.2450, 'Potato_Early_blight' 0.2089, whereas now these are significantly higher or 1.0000). The vast majority of classes maintain accuracies well above 0.95. The only notable exception is 'Corn (maize) Cercospora leaf spot Gray leaf spot' at 0.9709 and 'Corn (maize) Northern Leaf Blight' at 0.9442 and 'Potato healthy', which shows an accuracy of approximately 0.7097, indicating it is the single most challenging class for this model. This plot vividly illustrates the Custom ViT model's exceptional per-class performance across the dataset.



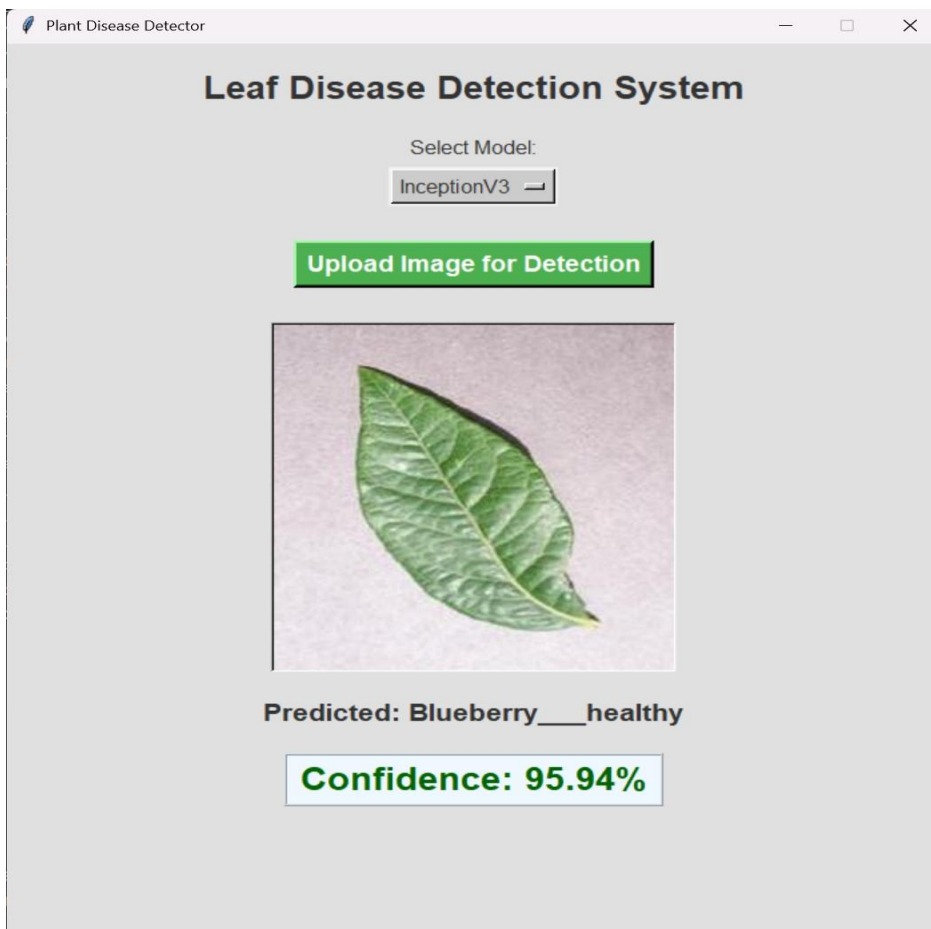
LLM Description Script (desc_llm.py)

This script is a standalone module responsible for generating textual descriptions using a Large Language Model. Its purpose is to provide dynamic, AI-generated, concise, and scientific descriptions of plant diseases based on their predicted labels, thereby enhancing the informational output of the main applications. Key functionality includes the `generate_llm_description(predicted_class_label)` function, which crafts a specific prompt for the LLM to generate a description focusing on symptoms and characteristics. It interacts with the Google Gemini API (gemini-2.0-flash) via an HTTP POST request and includes robust response parsing and error handling for network issues, API errors, or unexpected response formats. Dependencies include `requests` and `json`. Usage notes indicate it requires an active internet connection and access to the Gemini API (API key management is handled by the Canvas environment or needs to be configured for local execution). Its current status is fully implemented as a modular component, ready to be imported and utilized by other Python applications (like `app.py` and `backend_app.py`).

GUI Application (app.py)

Early App (app.py - Initial Version)

The initial version of the app.py script provided fundamental plant disease detection capabilities. In its early form, it was primarily designed for a single, pre-defined model, such as ResNet50 or InceptionV3, which necessitated manual modification of the script to switch between different model architectures. There was no direct support for a "Custom ViT" model at this stage. Furthermore, for disease descriptions, the application relied on a static, hardcoded dictionary (DISEASE_DESCRIPTIONS) embedded within the script, meaning descriptions were fixed and pre-written for each disease label. Regarding its graphical interface, the early app utilized standard tkinter widgets, which typically presented a more native but less visually modern appearance, with basic styling often achieved through direct fg (foreground) and bg (background) attributes. The application window was generally fixed in size, as indicated by app.resizable(False, False) preventing users from resizing it to better fit their screen or view more content, and text wrapping was also limited by fixed wraplength values on its labels. If any LLM description logic was considered, it would have been directly embedded or handled in a less decoupled manner within the main application file.

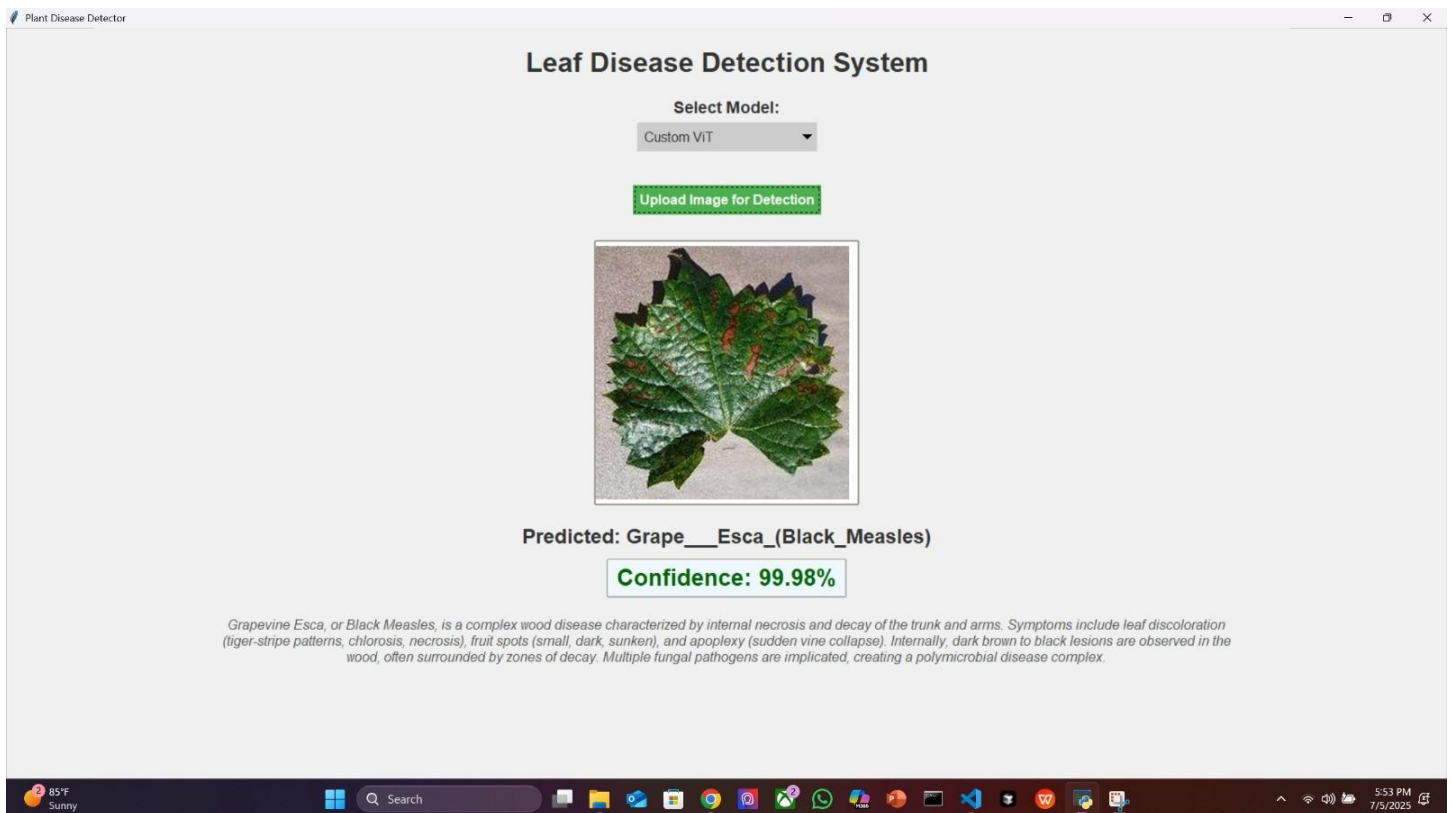


New GUI App (app.py - Current Version)

The current app.py represents a substantial upgrade, delivering enhanced flexibility, intelligence, and a significantly improved user experience. A major advancement is its multi-model support, allowing users to dynamically select between "ResNet50", "InceptionV3", a general "ViT" model (which is a pre-trained version), and a "Custom ViT" model (your fine-tuned version) directly from a dropdown menu within the interface. This functionality is robustly managed by a MODEL_PATHS_CONFIG dictionary and dynamic model loading functions, thereby eliminating the need for any code changes to switch between different prediction models.

A critical enhancement involves the dynamic LLM descriptions. Instead of relying on static lookups, the app now integrates with the desc_llm.py script to leverage a Large Language Model (LLM) (specifically Google's Gemini API) for dynamically generating concise and scientific descriptions for the predicted disease labels. This complex process runs asynchronously in a separate thread, which is crucial for preventing the GUI from freezing, and app.after() ensures the GUI is updated safely with the generated description once the LLM API call completes.

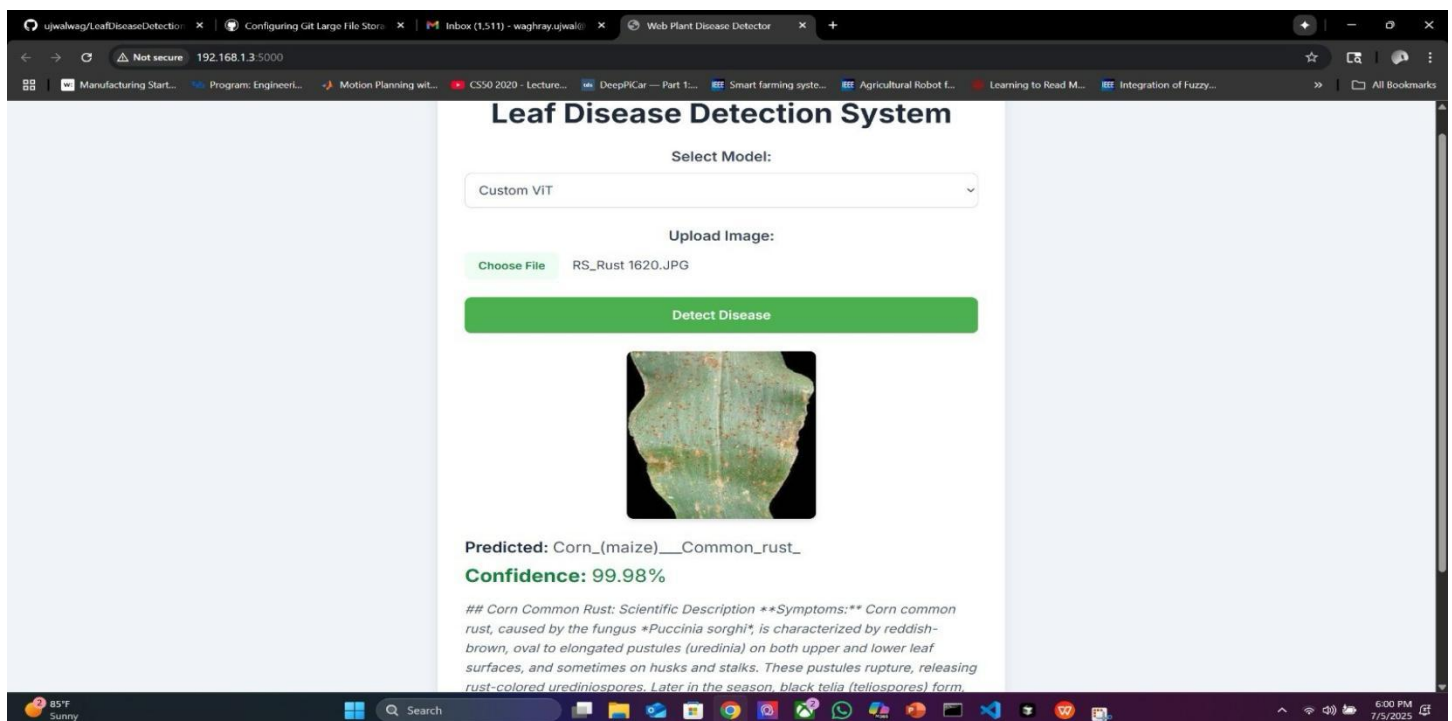
The GUI aesthetics have been significantly modernized through the adoption of tkinter.ttk and the application of a 'clam' theme. Custom ttk.Style configurations are meticulously used to define consistent fonts, colors, padding, and relief for all UI widgets, resulting in a cleaner, flatter, and more contemporary visual design. Furthermore, the application window is now resizable, empowering users to maximize or resize it according to their needs. Text labels displaying prediction results and disease descriptions also feature increased wraplength values, ensuring that longer texts wrap gracefully within the expanded window, thus making the full description visible.



Web Application Backend (backend_app.py) and Frontend (templates/index.html)

The web application comprises a backend and a frontend. The Web Application Backend (backend_app.py), built with Flask, serves as the server-side logic. It centralizes the loading of all specified deep learning models (ResNet50, InceptionV3, ViT, Custom ViT) into memory upon server startup for optimized performance. It exposes a /predict API endpoint that accepts image files and the selected model type via a POST request. This endpoint handles image preprocessing, runs inference using the loaded PyTorch models, and calls the desc_llm.py module to retrieve a dynamic disease description. The results (label, confidence, description) are returned as a JSON object to the frontend. The backend also implements robust error handling for various issues, returning appropriate HTTP status codes and informative messages. Its dependencies include flask, torch, torchvision, Pillow, os, io, transformers, and desc_llm, requiring correct file placement for templates and scripts.

Complementing the backend is the Web Application Frontend (templates/index.html), an HTML file providing the user interface. It features a responsive design using Tailwind CSS for optimal viewing across devices. Users can select a model from a dynamically populated dropdown, upload an image (with preview), and initiate asynchronous predictions via JavaScript's fetch API to the Flask backend. The webpage dynamically updates with the predicted disease label, confidence, and generated description, displaying loading indicators during processing. Its dependencies are HTML, CSS (Tailwind CSS), and JavaScript, and it must be located within a templates/ subdirectory relative to backend_app.py. This frontend currently offers a basic, functional, and modern web interface.



Project Overview

The Leaf Disease Detection System is structured with dedicated folders for scripts, models, raw and processed datasets, and results for comprehensive development and operation.

Data Preprocessing:

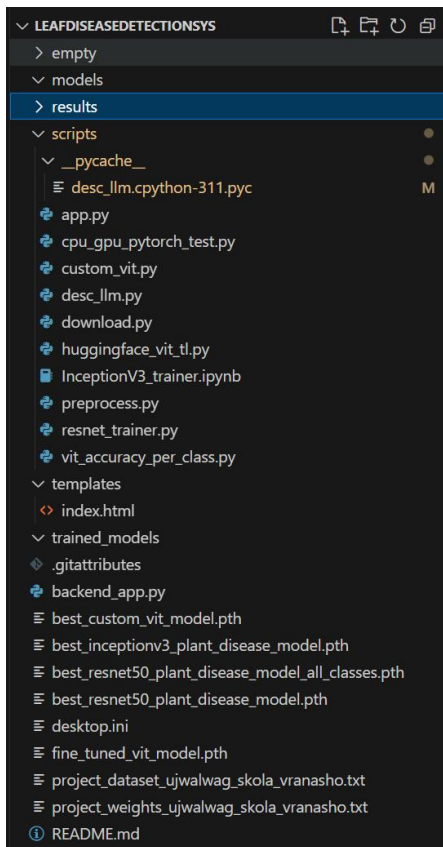
- Action: Organizes and resizes raw images for model training.
- Script: preprocess.py
- Location: Outputs to PlantVillage_Organized_Processed_Dataset.

Model Training:

- Action: Trains ResNet50, InceptionV3, and Vision Transformer deep learning models.
- Scripts: resnet_trainer.py, InceptionV3_trainer.ipynb, huggingface_vit_tl.py, custom_vit.py.
- Outputs: Trained models in models/ (e.g .pth files) and performance visuals in results/.

GUI Prediction:

- Action: Offers a user-friendly interface for real time disease detection.
- Script: app.py , web_app.py
- Functionality: Enables model selection, image upload, and displays predictions, incorporating LLM-generated descriptions.



Conclusion

This project demonstrates a complete pipeline for plant disease detection using computer vision and deep learning. From downloading and preparing the dataset to training the model and deploying a GUI application and a web application, every component works together to deliver an accurate and user-friendly disease detection system. It has practical applications in agriculture, allowing farmers and researchers to detect diseases early and take preventive measures.

Reference Links

Pretrained ViT : https://huggingface.co/wambugu71/crop_leaf_diseases_vit

Dataset: <https://github.com/spMohanty/PlantVillage-Dataset>

ResNet: [https://www.cv-](https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/He_Deep_Residual_Learning_CVPR_2016_paper.pdf)

[foundation.org/openaccess/content_cvpr_2016/papers/He_Deep_Residual_Learning_CVPR_2016_paper.pdf](https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/He_Deep_Residual_Learning_CVPR_2016_paper.pdf)

InceptionNet V3: <https://arxiv.org/abs/1512.00567>

Git - Who

```
(base) Vaibhavs-MacBook-Pro:LeafDiseaseDetectionSys vaibhavranshoor$ git-who -l
```

Author	Last Edit	Commits	Files	Lines (+/-)
Vaibhav Ranashoor	1 week ago	1	1	10,006 / 0
ujwalwag	18 min. ago	15	32	2,559 / 525
Vaibhav	39 min. ago	3	2	61 / 78
Ajaykola7	34 min. ago	3	1	1 / 1

```
(base) Vaibhavs-MacBook-Pro:LeafDiseaseDetectionSys vaibhavranshoor$ git-who tree -l
```

```
./.....Vaibhav Ranashoor (10,006 / 0)
├── results/.....ujwalwag (0 / 0)
│   ├── Figure_1_resnet1.png
│   ├── Figure_1_vit.png
│   ├── Figure_2_resnet1.png
│   ├── Figure_2_vit.png
│   ├── Figure_3_resnet1.png
│   ├── Figure_3_vit.png
│   ├── accuracy_per_class_vit.png
│   ├── confusion_matrix_vit.png
│   └── training_validation_metrics.png
├── scripts/.....Vaibhav Ranashoor (10,006 / 0)
│   ├── __pycache__/desc_llm.cpython-311.pyc.....ujwalwag (0 / 0)
│   ├── InceptionV3_trainer.ipynb
│   ├── app.py.....ujwalwag (458 / 134)
│   ├── cpu_gpu_pytorch_test.py.....Vaibhav (49 / 49)
│   ├── custom_vit.py.....ujwalwag (285 / 0)
│   ├── desc_llm.py.....ujwalwag (53 / 0)
│   ├── download.py.....ujwalwag (82 / 0)
│   ├── huggingface_vit_tl.py.....ujwalwag (235 / 0)
│   ├── preprocess.py.....ujwalwag (485 / 347)
│   ├── resnet_trainer.py.....ujwalwag (300 / 15)
│   └── vit_accuracy_per_class.py.....ujwalwag (68 / 0)
├── templates/index.html.....ujwalwag (170 / 1)
├── .gitattributes.....ujwalwag (1 / 0)
├── README.md.....ujwalwag (104 / 2)
├── backend_app.py.....ujwalwag (260 / 26)
├── best_custom_vit_model.pth.....ujwalwag (3 / 0)
├── best_inceptionv3_plant_disease_model.pth.....ujwalwag (0 / 0)
├── best_resnet50_plant_disease_model.pth.....ujwalwag (0 / 0)
├── best_resnet50_plant_disease_model_all_classes.pth...ujwalwag (0 / 0)
├── desktop.ini.....ujwalwag (4 / 0)
├── fine_tuned_vit_model.pth.....ujwalwag (0 / 0)
├── project_dataset_ujwalwag_skola_vranasho.txt.....ujwalwag (1 / 0)
└── project_weights_ujwalwag_skola_vranasho.txt.....ujwalwag (1 / 0)
(base) Vaibhavs-MacBook-Pro:LeafDiseaseDetectionSys vaibhavranshoor$
```

Team Member	UBIT	UBID	Contribution%
Ujwal Waghray	ujwalwag	50560587	40%
Vaibhav Vinay Ranashoor	vranasho	50596107	30%
Kola Siva Naga Sai Ajay	skola	50590721	30%